

Documentação de Microserviço

1. Descrição Funcional

O Order Service é um microserviço responsável pelo gerenciamento de pedidos realizados por clientes em uma plataforma de vendas baseada em arquitetura distribuída.

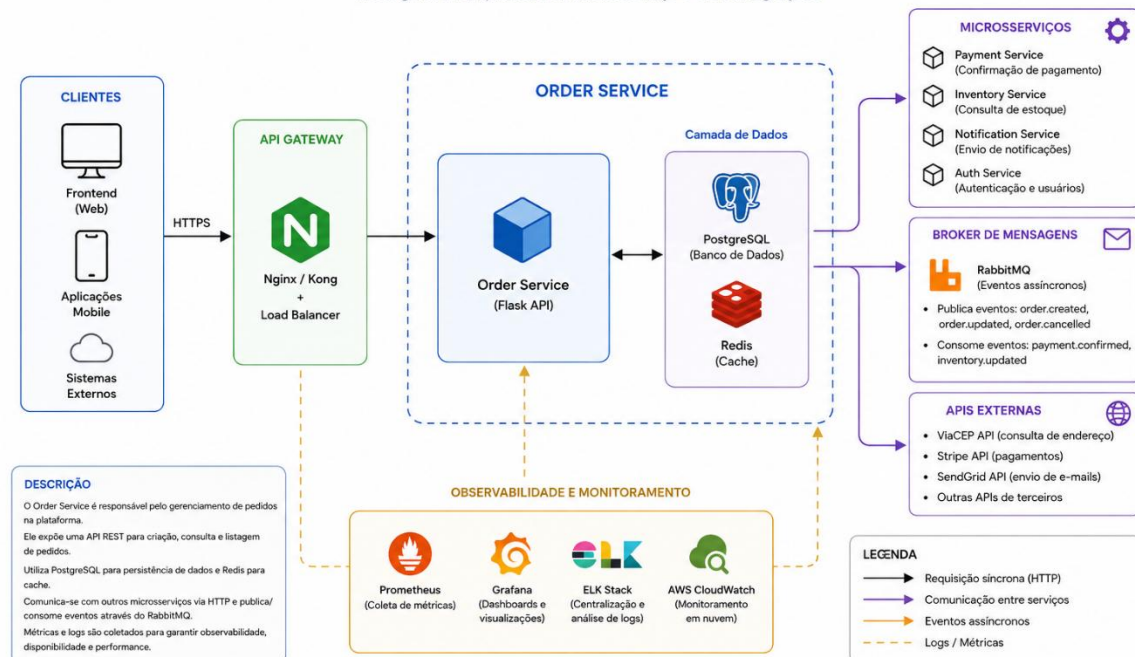
Principais responsabilidades:

- Criar pedidos;
- Consultar pedidos existentes;
- Buscar pedidos por identificador;
- Validar informações básicas do pedido;
- Registrar o status inicial do pedido;
- Disponibilizar dados para outros serviços da arquitetura.

Este microserviço segue o princípio de responsabilidade única, concentrando exclusivamente as operações relacionadas ao domínio de pedidos.

Diagrama Arquitetural

Visão geral da arquitetura do microserviço e suas integrações



2. Endpoints da API

Método	Endpoint	Descrição
GET	/orders	Lista todos os pedidos
POST	/orders	Cria um novo pedido
GET	/orders/{id}	Busca um pedido específico
GET	/health	Verifica a saúde do serviço

3. Exemplo de Requisição e Resposta

Criar Pedido- Requisição

POST /orders

```
``json
{
  "cliente": "Ana Souza",
  "produto": "Teclado Mecânico",
  "quantidade": 2
}
```

Resposta

```
``json
{
  "id": 1,
  "cliente": "Ana Souza",
  "produto": "Teclado Mecânico",
  "quantidade": 2,
  "status": "PROCESSANDO"
}
```

Buscar Pedido

GET /orders/1

Resposta

```
```json
{
 "id": 1,
 "cliente": "Ana Souza",
 "produto": "Teclado Mecânico",
 "quantidade": 2,
 "status": "PROCESSANDO"
}
```
```

4. Dependências Externas

O serviço Order Service foi projetado para futura integração com outros microsserviços:

- Payment Service: Validar confirmação de pagamento do pedido | REST API
- Inventory Service: Verificar disponibilidade de estoque | REST API
- Notification Service: Enviar notificações ao cliente | Assíncrono
- Auth Service | Validar autenticação e autorização | REST API

Fluxo de Integração

1. O pedido é criado no OrderFlow Service
2. O Inventory Service valida disponibilidade do produto
3. O Payment Service confirma o pagamento
4. O Notification Service envia confirmação ao cliente.

Banco de Dados

O microsserviço utiliza persistência de dados para armazenamento das informações dos pedidos.

PostgreSQL para armazenamento principal dos pedidos.
Redis como cache temporário para consultas frequentes.

Dados Persistidos

O banco armazena:

- Identificador do pedido;
- Dados do cliente;
- Produto solicitado;
- Quantidade;

- Status do pedido;
- Data de criação;
- Histórico de atualização.

Estratégia de Persistência

Cada microserviço possui banco de dados independente, seguindo o princípio de isolamento de dados em arquiteturas distribuídas.

4.3 Fila e Broker de Mensagens

Para comunicação assíncrona entre serviços, o sistema utiliza broker de mensagens.

RabbitMQ - Publicação e consumo de eventos.

Apache Kafka (planejado) - processamento de eventos em larga escala.

5. Responsáveis

Equipe Nexus responsável pelo desenvolvimento e manutenção do serviço.

Ponto Focal: Lais Gabriele

Contato: g.lais@aluno.ifsp.edu.br

6. Eventos

Eventos Publicados

| Evento | Descrição |
|-----------------|-------------------------------|
| order.created | Pedido criado com sucesso |
| order.updated | Alteração de status do pedido |
| order.cancelled | Cancelamento do pedido |

Eventos Consumidos

| Evento | Origem (Microserviço) |
|-------------------|-----------------------|
| payment.confirmed | Payment Service |
| inventory.updated | Inventory Service |

Benefícios da Comunicação Assíncrona

- Redução de acoplamento;
- Maior escalabilidade;
- Processamento resiliente;
- Melhor tolerância a falhas.

7. Procedimentos Básicos de Operação

Esta seção descreve os procedimentos necessários para execução, monitoramento e manutenção básica do Order Service em ambiente local ou containerizado.

7.1 Pré-requisitos

Antes de executar o microserviço, é necessário possuir os seguintes softwares instalados:

Python 3.11 ou superior

Docker 24+

Git Última versão

Pip Atualizado

7.2 Clonando o Repositório

Clone o projeto utilizando o Git:

```
git clone https://github.com/laisgsz/microservico_order_service.git
```

7.3 Executando Localmente

Instalação das Dependências

O projeto utiliza Flask como framework principal da API.

Execute o comando abaixo para instalar as dependências:

```
pip install -r requirements.txt
```

Inicialização da Aplicação

Após instalar as dependências, execute:

```
python app.py
```

Se a inicialização ocorrer corretamente, o terminal exibirá:

Running on <http://0.0.0.0:5000>

O serviço ficará disponível em:

<http://localhost:5000>

7.4 Executando com Docker

A aplicação pode ser executada em container Docker para facilitar padronização do ambiente.

Build da Imagem

Execute o comando:

```
docker build -t order-service
```

O Docker irá:

Baixar a imagem base Python;

Copiar os arquivos do projeto;

Instalar dependências;

Preparar o ambiente de execução.

Inicializar Container

```
docker run -d -p 5000:5000 --name order order-service
```

Parâmetros utilizados:

-d | Executa em background

-p 5000:5000 | Mapeia a porta local.

--name | Define nome do container.

Verificar Containers Ativos

`docker ps`

7.5 Verificação de Logs

Os logs permitem monitorar funcionamento da aplicação e identificar possíveis falhas.

Logs em Execução Local

Quando executado localmente, os logs são exibidos diretamente no terminal.

Exemplo:

```
[INFO] Pedido criado com sucesso  
[INFO] GET /orders/1
```

Logs no Docker

Para visualizar logs do container:

`docker logs order`

Logs em Tempo Real

`docker logs -f order`

A opção `-f` mantém acompanhamento contínuo das novas mensagens geradas pela aplicação.

Principais Eventos Registrados

O serviço registra:

- Inicialização da aplicação;
- Criação de pedidos;
- Consultas realizadas;
- Erros de validação;
- Falhas internas;
- Requisições HTTP recebidas.

7.6 Endpoint de Health Check

O serviço disponibiliza um endpoint específico para monitoramento de disponibilidade.

Endpoint

GET /health

URL completa:

<http://localhost:5000/health>

Objetivo

O endpoint permite:

- Verificar se a aplicação está operacional;
- Validar disponibilidade do serviço;
- Auxiliar ferramentas de monitoramento;
- Detectar falhas rapidamente.

Resposta Esperada

```
{  
  "status": "UP",  
  "service": "Order Service"  
}
```

Códigos HTTP Esperados:

| Código | Significado |
|---------------------------|----------------------|
| 200 OK | Serviço operacional |
| 500 Internal Server Error | Serviço indisponível |

7.7 Reinício do Serviço

O reinício pode ser necessário após alterações de código, falhas ou atualização de configuração.

Reinício Local

Interrompa a execução utilizando:

CTRL + C

Depois execute novamente:

```
python app.py
```

Reinício via Docker

```
docker restart order
```

Verificação Pós-Reinício

Após reiniciar:

1. Verificar se o container está ativo:

```
docker ps
```

2. Validar endpoint de health check:

GET <http://localhost:5000/health>

3. Conferir logs:

```
docker logs order
```

7.8 Procedimentos de Troubleshooting

Porta já utilizada

Erro: Port 5000 already in use

Solução:

Encerrar aplicação que utiliza a porta;

Alterar porta da aplicação.

Container encerrando automaticamente

Verificar logs:

docker logs order

Possíveis causas:

Erro de dependência;

Falha no código;

Porta ocupada.

Dependências não instaladas

Erro: ModuleNotFoundError

Solução: pip install -r requirements.txt

7.9 Boas Práticas Operacionais

As seguintes práticas são recomendadas:

- Utilizar containers para padronização;
- Monitorar logs continuamente;
- Implementar métricas e observabilidade;
- Realizar health checks periódicos;
- Utilizar variáveis de ambiente para configurações;
- Evitar informações sensíveis no código-fonte.

8. Monitoramento Futuro

Como evolução futura do projeto, está prevista a integração do microsserviço com ferramentas de monitoramento e observabilidade, com o objetivo de facilitar o acompanhamento do funcionamento da aplicação e melhorar a identificação de possíveis falhas.

O Prometheus poderá ser utilizado para coletar métricas importantes da API, como número de requisições realizadas, tempo médio de resposta e quantidade de erros registrados durante a execução do serviço.

O Grafana será responsável pela criação de dashboards e visualizações gráficas dessas métricas, permitindo um acompanhamento mais simples e intuitivo do comportamento do microsserviço em tempo real.

Também existe o planejamento de utilizar a ELK Stack (Elasticsearch, Logstash e Kibana) para centralização e análise de logs. Com isso, será possível rastrear eventos da aplicação, investigar erros com mais facilidade e melhorar o processo de troubleshooting.

Em ambientes hospedados na nuvem, o AWS CloudWatch poderá ser integrado para monitoramento da infraestrutura e geração de alertas automáticos em casos de indisponibilidade ou comportamento anormal do sistema.

Essas ferramentas contribuirão para aumentar a observabilidade, confiabilidade e estabilidade do microsserviço, além de facilitar futuras manutenções e melhorias da aplicação.